Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Multi-Drone Search & Rescue for Dynamic Disaster Response Using Graph-Based Path Planning and Priority Task Allocation

Team Name: *Team #*	*AeroResQ* *DSCPP-III-2026-T290*
Team member 1 (Team Lead) *Fangalia, Aarav : 2510370262 :* *aaravfangalia@gmail.com*	**FANGALIA, AARAV** 91496 08084 *aaravfangalia@gmail.com* 
Team member 2 *Kothari, Jayna : 2510380106 :* *jaynakothari944@gmail.com*	**KOTHARI, JAYNA** 96503 02552 *jaynakothari944@gmail.com* 
Team member 3 *Singh, Hrimandeep : 2510380210 :* *hrimandeep@gmail.com*	**SINGH, HRIMANDEEP** 85860 01082 *hrimandeep@gmail.com* 
Team member 4 *Singhal, Raghav : 2510340011 :* *rsinghal249@gmail.com*	**SINGHAL, RAGHAV** 85277 01488 *rsinghal249@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

Search-and-rescue after an earthquake, flood or landslide involves many locations, different urgency levels, limited drones and changing access conditions. Our project, *AeroResQ*, is a simulation-based coordination system that decides, repeatedly and explainably, which rescue mission is served first, which UAV serves it, and which route that UAV follows. The disaster zone is modelled as a weighted undirected graph stored as an adjacency list. Rescue requests are kept in a priority queue so urgent missions are considered first. A Coordinator selects the eligible UAV with the lowest route cost, using Dijkstra's algorithm for weighted routes and BFS for reachability checks, and respects availability and a simplified battery rule (10% battery per route-cost unit). The system also reacts to four dynamic events: low battery, UAV failure, blocked route and a new urgent mission, by re-planning the affected missions.

The project is deliberately scoped as a student-level C/C++ simulation, not a flight controller; it connects graphs, shortest paths, priority queues and object-oriented design in one workflow.

In Phase II the core engine, the dynamic event module and the five analytic test scenarios have been designed and documented in an IEEE-style research paper, and the remaining work is dashboard/console polishing, extended testing and final integration.

Updated Project Approach and Architecture (2 pts)

Design. The architecture follows a vertical pipeline:

Mission Input $\rightarrow$ *Priority Queue* $\rightarrow$ *Coordinator* $\rightarrow$ *Route Planner (Dijkstra + BFS)* $\rightarrow$ *UAV State (battery + availability)* $\rightarrow$ *Assignment & Execution.*

An Event Handler feeds failures, low battery, blocked routes and urgent missions back into the Coordinator, which triggers re-planning. Keeping routing, UAV state and allocation policy in separate modules lets each be tested independently.

Compared with Phase I, the design is now fully formalised: eligibility is defined as

$$\textit{available} \wedge \textit{reachable} \wedge \textit{battery} \geq 10\times \textit{route cost},$$

and the Coordinator assigns each mission (highest priority first) to the eligible UAV with the smallest cost, breaking ties by higher battery.

Data structures. The map is a six-node, eight-edge weighted graph in an adjacency list (each undirected edge stored twice; a blocked route is a flag on the edge, so no rebuild is needed).

Missions use a binary-heap priority queue with a sequence number to keep arrival order among equal priorities.

Implementation. C is used for data-structure logic (graph, edges, traversals) and C++ for object-oriented modelling (`UAV`, `Mission`, `Coordinator`, event handling).

The interface is a console/menu program.

Communication and libraries. Being a single-process simulation, modules communicate through direct function and method calls; no network protocol is used. Libraries are limited to the C/C++ standard library (`<queue>`, `<vector>`, `<string>`, `<limits>`).

Events are injected deterministically so every scenario is repeatable.

A lightweight dashboard (UAV fleet, mission queue, route view, events, activity log) is planned as a visual layer; ArduPilot SITL remains a future extension.

Tasks Completed (7 pts)

Task Completed	Team Member
Finalised problem statement, formal model and eligibility/assignment rule; Coordinator allocation loop (Algorithm 2)	Aarav Fangalia
Weighted-graph adjacency list in C; BFS reachability; heap-based Dijkstra with predecessor tracking (verified by hand trace)	Jayna Kothari
Priority-queue mission handler with sequence numbers; UAV and Mission classes; dynamic event handler (Algorithm 3) for low battery, failure, blocked route and urgent mission	Raghav Singhal
Five analytic test scenarios and verification table; figures (graph, Dijkstra tree, cost and success-rate charts); dashboard concept; IEEE-style research paper	Hrimandeep Singh
GitHub repository set-up, project structure and commit history	Aarav Fangalia, Hrimandeep Singh

Challenges/Roadblocks (7 pts)

1. **Stale routes after graph changes.** A blocked edge (4–5) raised the best route from node 1 to node 6 from 7 to 8, so cached costs gave wrong battery decisions.
   Solution: Routes are never cached; Dijkstra and BFS are re-run for every affected mission after each event.
2. **Interacting events.** A low-battery report and an urgent mission can arrive in the same cycle.
   Solution: A fixed re-evaluation order (apply event, find affected missions, check priority, recompute) and inserting the urgent mission first so eligibility uses updated battery values.
3. **Starvation of low-priority missions.** When UAVs are fewer than requests, M-103 stays pending.
   Solution: Such missions remain pending and are retried each cycle; an optimal-assignment comparison is planned.
4. **Greedy allocation is not globally optimal.** It matched the optimum (total cost 8) in the example but offers no guarantee in general.
   Solution: Document the limitation and compare against an exhaustive assignment on small cases.
5. **Realistic battery model.** The 10%-per-cost rule is a simulation rule.
   Solution: Keep it parameterised (β) so it can be changed later.
6. **Mixing C and C++.** Sharing C structures with C++ classes needed careful interface design.
   Solution: Wrap the C graph behind `extern "C"` and use it from Coordinator.
7. **Team coordination.** Parallel work on shared files risked merge conflicts.
   Solution: Module-wise ownership and small, frequent commits.

Tasks Pending (7 pts)

Task Pending	**Team Member (to complete the task)**
Complete the console/menu interface (add mission, change battery, block route, fail UAV, show assignments)	Hrimandeep Singh
Run all five scenarios as an executed program and compare printed output with the hand-derived table	Jayna Kothari, Raghav Singhal
Randomly generated larger graphs; measure processing time and assignment success rate as missions per UAV grow	Jayna Kothari
Optimisation: single Dijkstra run from the mission destination to get costs for all UAVs	Aarav Fangalia
Mission completion and return-to-base logic; automatic reassignment after a failed mission	Aarav Fangalia, Hrimandeep Singh
Dashboard prototype (fleet, queue, route view, event panel, activity log)	Raghav Singhal
Final report, demo preparation and (optional) ArduPilot SITL study	All members

Project Outcome/Deliverables (2 pts)

- A working C/C++ prototype that coordinates simulated drones.
- Graph-based map with Dijkstra route costs and BFS reachability checks.
- Priority-based rescue mission handling.
- Drone selection using availability, battery and route cost.
- Dynamic reallocation on low battery, UAV failure, blocked route and urgent missions.
- Test results showing which drone was assigned, the route cost, and the missions left pending.
- An IEEE-style research paper and a lightweight dashboard concept.

Progress Overview (2 pts)

Roughly **70%** of the project is complete. The design, algorithms, class model, event-handling logic, analytic evaluation and research paper are finished.

On schedule: graph, BFS, Dijkstra, priority queue and Coordinator logic.

Ahead of schedule: the research paper, complexity analysis and figures, completed earlier than planned.

Behind schedule: the polished console interface and the executed-program verification of all scenarios, which need more integration time.

The dashboard is still at concept stage, and ArduPilot SITL is optional and will only be attempted if time permits.

Codebase Information (2 pts)

Repository: `https://github.com/hrimandeepsingh/multi-drone-search-rescue-PBL`
Branch: `main`
Important commits:

- Initial project structure and README.
- Adjacency-list graph with edge costs and blocked-edge flag.
- BFS reachability and heap-based Dijkstra with predecessor tracking.

- Priority-queue mission handler and `UAV`/`Mission` classes.
- Coordinator allocation loop (Algorithm 2).
- Dynamic event handling (Algorithm 3) and scenario test cases.

(Full history: "View Commits" on the PBL Connect portal.)

Testing and Validation Status (2 pts)

Test Type	**Status (Pass/Fail)**	**Notes**
Dijkstra trace (source node 1)	Pass	Distances 4, 3, 4, 6, 7 to nodes 2–6; route 1-3-4-5-6, cost 7
BFS reachability	Pass	Node 6 reachable; fewest-edge route differs from cheapest route, so BFS used only for reachability
Scenario: normal allocation	Pass	M-101→C, M-102→B, M-103→A; total cost 8; 3 of 3 served
Scenario: low battery (C at 5%)	Pass	C rejected for M-101; B takes it; 3 of 3 served
Scenario: UAV-C failure	Pass	M-103 stays pending; 2 of 3 served
Scenario: edge 4–5 blocked	Pass	Cost to node 6 rises 7→8; total cost 10; BFS still reaches node 6
Scenario: urgent M-104 (P4)	Pass	M-104 served first; M-103 pending; 3 of 4 served
Large random graphs / timing	Pending	Planned for Phase III

Deliverables Progress (2 pts)

Deliverable	**Status**
Graph-based map and route calculation (Dijkstra/BFS)	Completed
Priority-based mission handling	Completed
Drone selection (availability, battery, cost)	Completed
Reallocation on failure / change of mission	Completed
Test scenarios and results	In progress (analytic done; program runs being finalised)
Console/menu interface	In progress
Research paper	Completed
Dashboard prototype	Pending